Projet de classification de feuilles d'arbres

Réseaux de neurones convolutifs (CNN) appliqués à la reconnaissance d'espèces

Mehdaoui Hafsa

ISN 2A FISE 2026 — filière SA

Méthodes d'apprentissage

30 juin 2026

Résumé

Ce compte rendu présente la conception et l'évaluation de réseaux de neurones convolutifs pour la classification d'images de feuilles d'arbres. Deux bases de données sont étudiées : une base réduite de **4 espèces** et la base complète de **32 espèces**. Pour chaque base, on cherche une architecture atteignant une précision de validation supérieure à **75 %**. La démarche, identique d'un cas à l'autre, suit le schéma *Objectif / Démarche / Résultats et interprétation*. Les résultats obtenus (98,4 % sur 4 espèces, 96,8 % sur 32 espèces) montrent qu'une architecture peu profonde suffit pour un problème simple, tandis qu'un réseau plus profond devient indispensable lorsque le nombre de classes augmente. Une expérience de régularisation (data augmentation et dropout) est également menée afin d'étudier le contrôle du surapprentissage.

Table des matières

1 Introduction **2**

2 Cadre théorique et méthodologie **2**
2.1 Construction de la base de données 2
2.2 Architecture d'un réseau convolutif 2
2.3 Apprentissage et évaluation 3

3 Exercice 1 : classification de 4 types de feuilles **3**
3.1 Objectif 3
3.2 Démarche 3
3.3 Résultats et interprétation 4

4 Exercice 2 : classification de 32 types de feuilles **5**
4.1 Objectif 5
4.2 Démarche 6
4.3 Résultats et interprétation 6
4.4 Expérience complémentaire : régularisation 8

5 Synthèse comparative **9**

6 Conclusion **10**

1 Introduction

La reconnaissance automatique d'espèces végétales à partir de photographies de feuilles est un problème classique de vision par ordinateur. Contrairement à une base prête à l'emploi telle que MNIST, les données fournies ici sont brutes : chaque image se trouve dans un dossier dont le *nom* constitue l'étiquette de classe. La première étape consiste donc à transformer cette arborescence de dossiers en une base de données exploitable.

L'objectif du projet est double :

— construire une architecture capable de classifier correctement une base réduite de 4 types de feuilles parmi les 32 disponibles ;
— construire une architecture adaptée à la base complète des 32 types.

Dans les deux cas, le critère de réussite imposé par l'énoncé est une précision supérieure à 75 %, en veillant à éviter le surapprentissage (*overfitting*). Ce document décrit d'abord le cadre théorique et la méthodologie commune, puis traite successivement les deux études de cas avant de conclure par une synthèse comparative.

2 Cadre théorique et méthodologie

2.1 Construction de la base de données

La base est créée à partir des dossiers d'images grâce à la fonction `keras.utils.image_dataset_from_direct`
Les principaux paramètres utilisés sont les suivants :

— `labels="inferred"` : les étiquettes sont déduites du nom des sous-dossiers ;
— `label_mode="categorical"` : chaque étiquette est codée par un vecteur *one-hot* ;
— `color_mode="rgb"` : les images sont traitées en couleur (3 canaux) ;
— `validation_split=0.1` et `subset="both"` : 90 % des images servent à l'entraînement, 10 % à la validation ;
— `image_size=(size, size)` : redimensionnement de toutes les images à une taille commune.

2.2 Architecture d'un réseau convolutif

Un réseau convolutif enchaîne deux phases : une phase d'*extraction de caractéristiques* (blocs de convolution) et une phase de *classification* (couches entièrement connectées).

Normalisation. Les valeurs de pixels, comprises entre 0 et 255, sont ramenées dans l'intervalle $[0, 1]$ par la couche `Rescaling` :

$$x' = \frac{x}{255}. \tag{1}$$

Convolution. Chaque filtre K de taille 3×3 balaie l'image I et produit une carte d'activation par produit de convolution :

$$(I * K)(i, j) = \sum_{m} \sum_{n} I(i+m,\, j+n)\, K(m, n). \tag{2}$$

L'option `padding="same"` ajoute une bordure de zéros afin que la carte de sortie conserve les mêmes dimensions spatiales que l'entrée ; cela permet d'empiler plusieurs blocs sans réduire trop vite la taille de l'image.

Activation ReLU. Après chaque convolution, une non-linéarité est appliquée :

$$f(x) = \max(0,\, x). \tag{3}$$

Mise en commun (Max pooling). Chaque bloc 2×2 est remplacé par sa valeur maximale, ce qui divise par deux la résolution spatiale et réduit le nombre de paramètres :

$$y(i,j) = \max_{(p,q)\in 2\times 2} x(2i+p,\, 2j+q). \tag{4}$$

Classification. Après aplatissement (`Flatten`), une couche dense `softmax` fournit une distribution de probabilité sur les C classes :

$$\sigma(z)_c = \frac{e^{z_c}}{\sum_{k=1}^{C} e^{z_k}}. \tag{5}$$

2.3 Apprentissage et évaluation

Le modèle est optimisé (optimiseur `adam`) en minimisant l'entropie croisée catégorielle entre l'étiquette réelle y et la prédiction $\hat{y}$:

$$\mathcal{L} = -\sum_{c=1}^{C} y_c \, \log(\hat{y}_c). \tag{6}$$

La qualité est mesurée par la précision (*accuracy*), proportion d'images correctement classées :

$$\text{précision} = \frac{\text{nombre de prédictions correctes}}{\text{nombre total d'images}}. \tag{7}$$

On distingue la précision d'entraînement (`accuracy`) de la précision de validation (`val_accuracy`). Un écart important entre les deux, accompagné d'une perte de validation élevée, signale un sur-apprentissage.

3 Exercice 1 : classification de 4 types de feuilles

3.1 Objectif

On cherche l'architecture la plus simple permettant de dépasser 75 % de précision sur une base de 4 espèces : *bitter orange*, *rose*, *sweet potato* et *vieux garçon*. La base contient 620 images (558 pour l'entraînement, 62 pour la validation). La figure 1 en montre un échantillon.

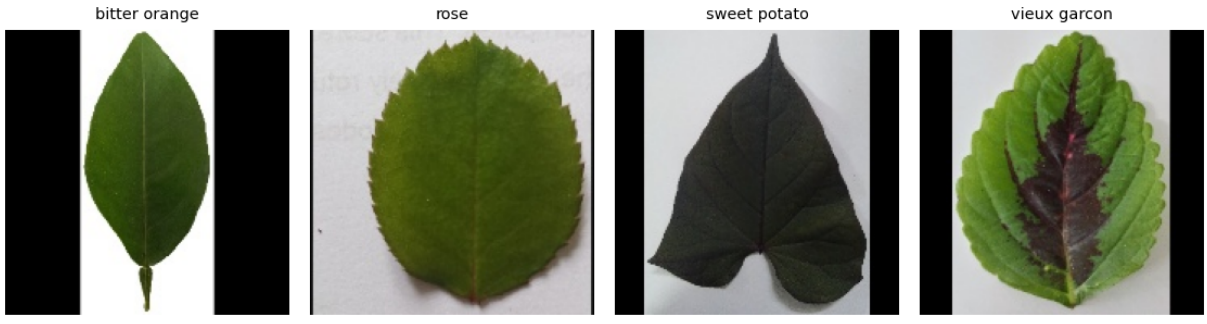


Figure 1 – Un exemple de feuille pour chacune des 4 classes.

3.2 Démarche

On part d'une architecture minimale : un seul bloc convolutif (10 filtres 3×3, `padding="same"`), suivi d'un *max pooling* 2×2, d'un aplatissement et d'une couche `softmax`. Les images sont redimensionnées en 28×28. À partir de cette configuration de référence, on fait varier **un paramètre à la fois** : le nombre d'*epochs* puis le nombre de filtres, afin d'observer leur influence.

3.3 Résultats et interprétation

Influence du nombre d'epochs. Le tableau 1 reporte les performances de l'architecture de référence (10 filtres) pour différentes durées d'entraînement.

epochs	accuracy	val_accuracy	loss	val_loss
5	0,8620	0,9516	0,5754	0,4666
15	0,9247	0,9516	0,2320	0,2480
30	0,9821	0,9839	0,1118	0,1498

Table 1 – Effet du nombre d'epochs (1 bloc, 10 filtres, taille 28).

Dès 5 epochs, la précision de validation atteint 95 %, déjà bien au-delà du seuil exigé. En prolongeant l'entraînement, les deux courbes continuent de progresser *conjointement* et la perte décroît régulièrement (figure 2). Fait notable : la précision de validation reste supérieure ou égale à la précision d'entraînement, signe qu'il n'y a *aucun* surapprentissage. Cela s'explique par la simplicité du problème (4 classes seulement, images peu bruitées).

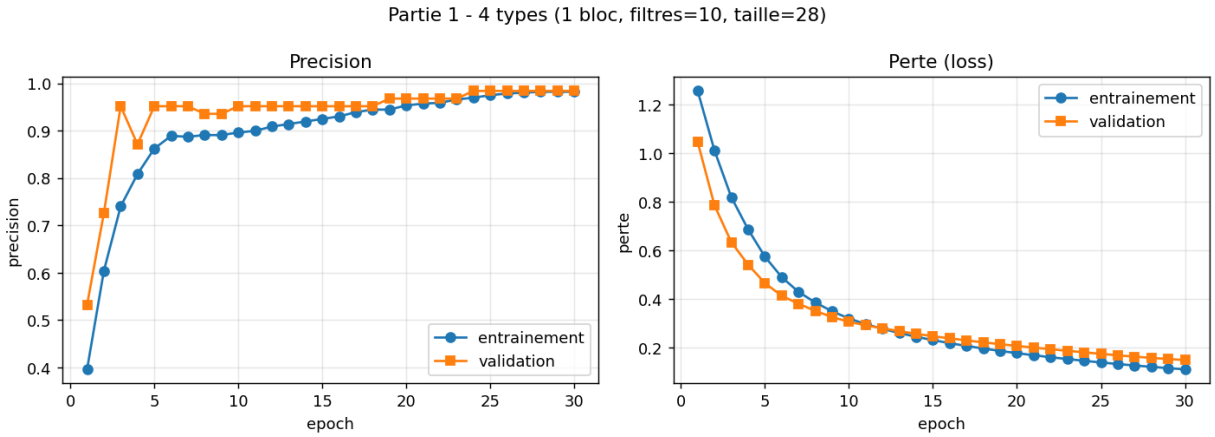


Figure 2 – Évolution de la précision et de la perte (30 epochs) pour l'exercice 1.

Influence du nombre de filtres. Le tableau 2 montre l'effet du nombre de filtres du bloc convolutif, à 5 epochs.

filtres	accuracy	val_accuracy	loss	val_loss
1	0,5161	0,5968	1,1703	1,1108
10	0,8620	0,9516	0,5754	0,4666
50	0,9014	0,9355	0,3457	0,2965

Table 2 – Effet du nombre de filtres (1 bloc, taille 28, 5 epochs).

Avec un seul filtre, le réseau ne dispose pas d'assez de capacité pour distinguer les classes (60 % en validation). Dès 10 filtres, la performance bondit à 95 % ; au-delà (50 filtres), le gain devient marginal pour un coût de calcul plus élevé. Le choix de 10 filtres constitue donc un bon compromis.

Matrice de confusion. La figure 3 confirme l'excellente séparation des classes : la quasi-totalité des prédictions se situe sur la diagonale.

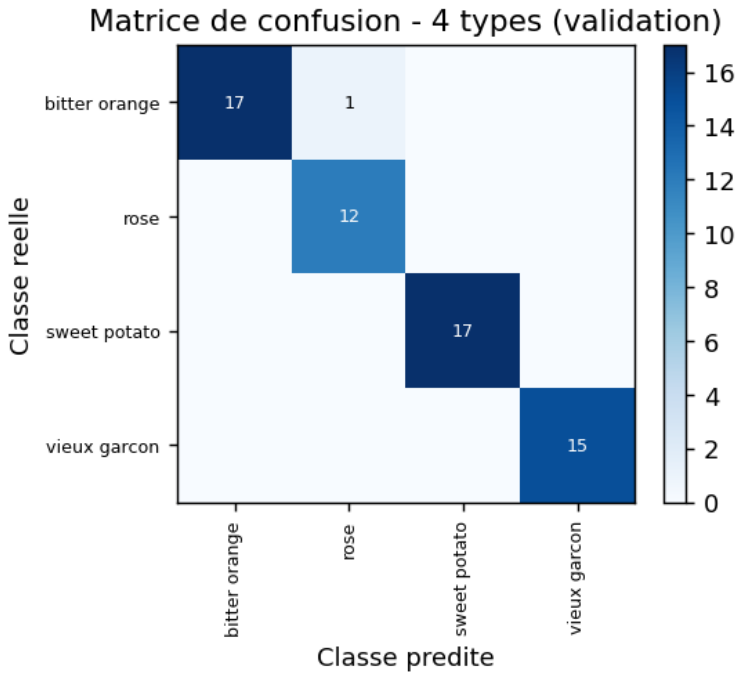


Figure 3 – Matrice de confusion sur l'ensemble de validation (4 classes).

Conclusion de l'exercice. Une architecture à **un seul bloc convolutif** (10 filtres, taille 28) entraînée 30 epochs atteint **98,39 %** de précision de validation. Pour un problème aussi restreint, augmenter la profondeur du réseau est inutile.

4 Exercice 2 : classification de 32 types de feuilles

4.1 Objectif

On traite désormais la base complète des 32 espèces (4955 images : 4460 pour l'entraînement, 495 pour la validation), illustrée figure 4. Le problème est beaucoup plus difficile : plus de classes, plus de variabilité intra-classe et des espèces visuellement proches.

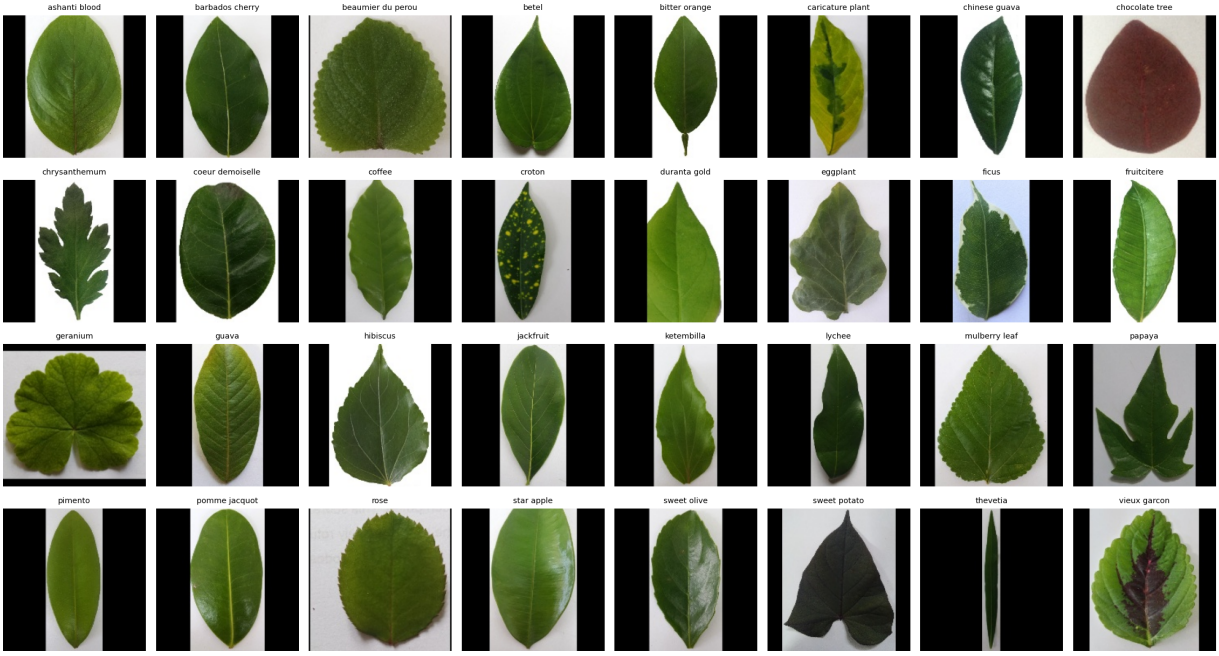


Figure 4 – Un exemple de feuille pour chacune des 32 classes.

4.2 Démarche

On vérifie d'abord que l'architecture peu profonde de l'exercice 1 est insuffisante, puis on *approfondit* progressivement le réseau. L'architecture finale retenue comporte **quatre blocs convolutifs** de profondeur croissante (10, 30, 250 puis 500 filtres), chacun suivi d'un *max pooling*. Le `padding="same"` est conservé pour éviter que l'image ne soit réduite à une taille 1×1 avant le dernier bloc. La taille des images est portée à 56×56 afin de préserver davantage d'information. On étudie ensuite l'effet du nombre d'epochs et du nombre de filtres, puis on mène une expérience complémentaire de *régularisation* (data augmentation et dropout) pour analyser le surapprentissage.

4.3 Résultats et interprétation

Un réseau peu profond est insuffisant. En appliquant directement l'architecture de l'exercice 1 (1 bloc, taille 28, 5 epochs), on obtient seulement **63,0 %** de précision de validation (tableau 3) : le modèle est trop superficiel pour capturer la complexité des 32 classes. Le passage à un réseau profond (4 blocs, taille 56) entraîné suffisamment longtemps change radicalement le résultat.

Architecture	accuracy	val_accuracy	loss	val_loss
1 bloc (taille 28, 5 ep.)	0,6224	0,6303	1,4491	1,4663
4 blocs (taille 56, 30 ep.)	1,0000	0,9677	0,0003	0,1435

Table 3 – Comparaison d'un réseau peu profond et du réseau profond retenu.

Influence du nombre d'epochs (réseau profond). Le tableau 4 montre la progression de l'architecture à 4 blocs.

epochs	accuracy	val_accuracy	loss	val_loss
5	0,8641	0,7980	0,4330	0,6450
8	0,9343	0,8525	0,2075	0,4638
30	1,0000	0,9677	0,0003	0,1435

Table 4 – Effet du nombre d'epochs (4 blocs, 10 filtres, taille 56).

Dès 5 epochs, le seuil de 75 % est franchi (79,8 %). En prolongeant l'entraînement, la précision de validation continue de croître pour atteindre **96,77 %** à 30 epochs. La figure 5 montre toutefois un phénomène caractéristique : la précision d'entraînement atteint 100 % et la perte d'entraînement tend vers zéro, alors que la perte de validation se stabilise autour de 0,14. Le modèle *mémorise* donc parfaitement la base d'entraînement (surapprentissage visible sur la perte), même si, sur cet ensemble de validation, la précision reste excellente.

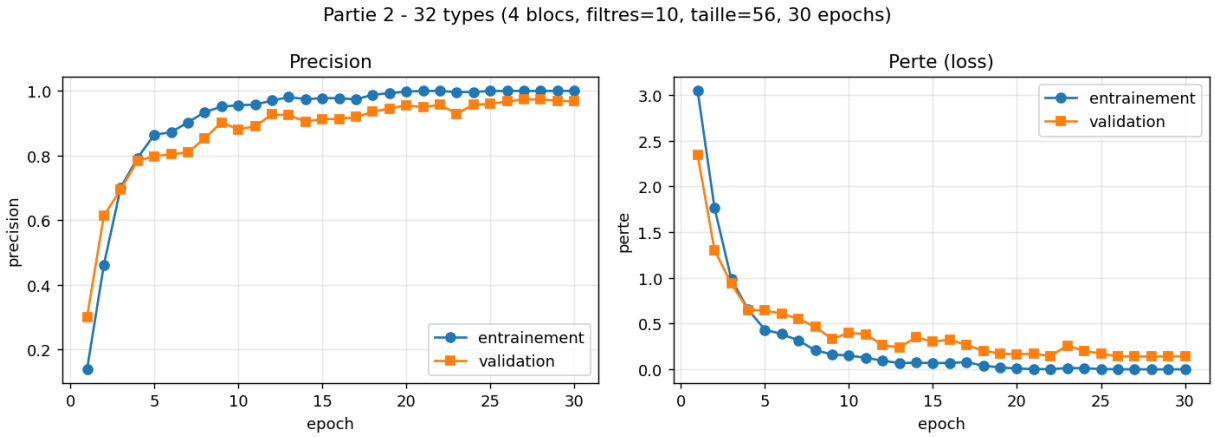


Figure 5 – Évolution de la précision et de la perte (30 epochs) pour l'exercice 2.

Influence du nombre de filtres (réseau profond). Le tableau 5 fait varier le nombre de filtres du premier bloc (les suivants étant proportionnels), à 5 epochs.

filtres	accuracy	val_accuracy	loss	val_loss
5	0,7399	0,7475	0,8430	0,8368
10	0,8641	0,7980	0,4330	0,6450
15	0,8377	0,8000	0,4928	0,6301

Table 5 – Effet du nombre de filtres (4 blocs, taille 56, 5 epochs).

Avec 5 filtres, le réseau passe tout juste sous le seuil en entraînement et reste fragile. À partir de 10 filtres, la précision de validation dépasse confortablement 75 %. Le passage à 15 filtres n'apporte qu'un gain négligeable. On retient donc **10 filtres** pour le premier bloc dans l'architecture finale.

Matrice de confusion. La figure 6 montre une diagonale nettement dominante. Les rares confusions concernent des espèces visuellement similaires, ce qui est attendu pour un problème à 32 classes.

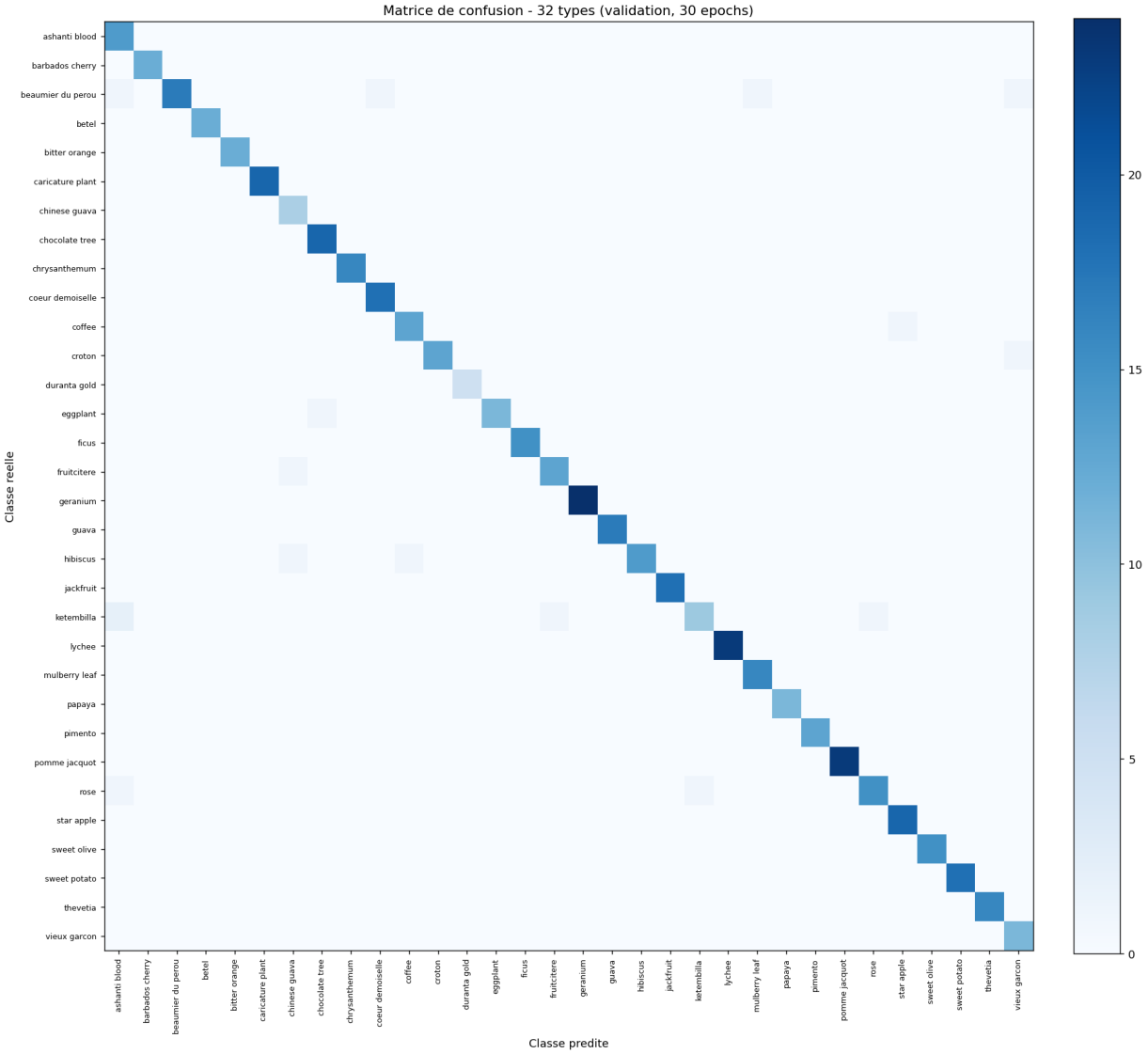


Figure 6 – Matrice de confusion sur l'ensemble de validation (32 classes, 30 epochs).

4.4 Expérience complémentaire : régularisation

La perte d'entraînement quasi nulle (tableau 4) révèle que le réseau *mémorise* la base d'apprentissage. Une parade classique consiste à *régulariser* le modèle. On ajoute donc :

- de la **data augmentation** (`RandomFlip`, `RandomRotation`, `RandomZoom`) appliquée aux seules images d'entraînement ;
- une couche de **dropout** (taux 0,4) avant la couche de sortie, qui désactive aléatoirement une partie des neurones à chaque itération.

Le tableau 6 compare, à nombre d'epochs égal (30), le réseau profond avec et sans régularisation.

Modèle (30 epochs)	**accuracy**	**val_accuracy**	**loss**	**val_loss**
Sans régularisation	1,0000	0,9677	0,0003	0,1435
Avec augmentation + dropout	0,9413	0,8384	0,1715	0,4334

Table 6 – Effet de la régularisation sur le réseau profond (4 blocs, taille 56, 30 epochs).

La régularisation joue bien son rôle : la précision d'entraînement chute de 100 % à 94 % et la

perte d'entraînement n'est plus quasi nulle, signe que le modèle ne mémorise plus la base. La figure 7 montre d'ailleurs des courbes d'entraînement et de validation beaucoup plus resserrées. En revanche, sur cette base, la régularisation *n'améliore pas* la précision de validation (84 % contre 96 %). Cela s'explique par le fait que l'ensemble de validation, issu du même tirage que l'entraînement, est très proche des données d'apprentissage : le modèle non régularisé généralise donc déjà très bien, et l'augmentation ne fait que compliquer l'apprentissage sans bénéfice mesurable ici. La régularisation resterait néanmoins précieuse face à des images réellement nouvelles (conditions de prise de vue différentes).

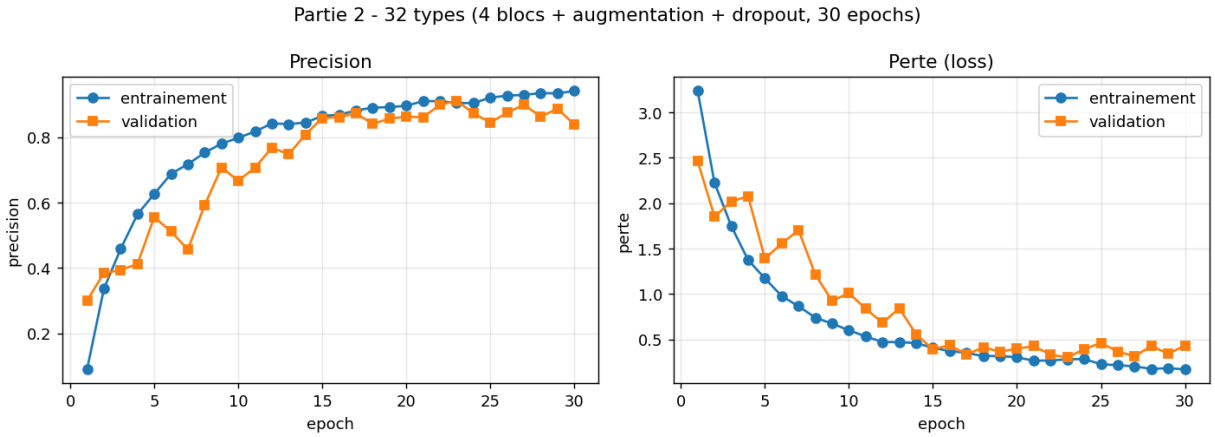


Figure 7 – Réseau profond *avec* régularisation : précision et perte (30 epochs).

Conclusion de l'exercice. L'architecture retenue est le réseau à **quatre blocs convolutifs** (taille 56, 10 filtres au premier bloc) entraîné **30 epochs**, qui atteint **96,77 %** de précision de validation. La régularisation, bien qu'efficace pour limiter la mémorisation, n'apporte pas de gain de précision sur cette base et n'est donc pas conservée dans le modèle final.

5 Synthèse comparative

Le tableau 7 récapitule les deux architectures retenues.

	Exercice 1 (4 types)	**Exercice 2 (32 types)**
Nombre de classes	4	32
Images (train / val)	558 / 62	4460 / 495
Taille des images	28×28	56×56
Blocs convolutifs	1	4
Filtres	10	10, 30, 250, 500
Epochs	30	30
val_accuracy	**0,9839**	**0,9677**

Table 7 – Synthèse des architectures retenues pour les deux bases.

Cette comparaison met en évidence un principe central : la complexité de l'architecture doit être adaptée à celle du problème. Un réseau trop simple ne peut pas représenter 32 classes, tandis qu'un réseau trop profond serait superflu (et plus coûteux) pour 4 classes.

6 Conclusion

Ce projet a permis de construire, pour deux bases de données de feuilles d'arbres, des réseaux convolutifs dépassant largement le seuil de 75 % imposé : 98,4 % pour 4 espèces et 96,8 % pour 32 espèces. La méthodologie adoptée — partir d'une architecture simple, puis faire varier un paramètre à la fois (epochs, filtres, profondeur, taille d'image) — s'est révélée efficace pour comprendre le rôle de chaque hyperparamètre.

On retient principalement qu'une architecture adaptée à une base ne l'est pas nécessairement à une autre : la base à 4 classes se résout avec un unique bloc convolutif, alors que la base à 32 classes nécessite un réseau profond et des images de plus grande taille. L'expérience de régularisation (*dropout* et *data augmentation*) a montré qu'elle réduit effectivement la mémorisation de la base d'entraînement, mais qu'elle n'améliore pas la précision de validation lorsque celle-ci est très proche des données d'apprentissage. Pour aller plus loin, il serait pertinent d'évaluer ces modèles sur un ensemble de test réellement indépendant (images prises dans d'autres conditions), cas où la régularisation devrait pleinement révéler son intérêt.